Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Лабораторна робота № 1

з дисципліни «Системне програмування»

на тему «Налаштування середовища розробки та запуск ExpoGuard у Wokwi»

Виконав:
студент групи ІО-32
Бурлій Андрій Євгенійович

Перевірив:
Гордієнко Нікіта Юрійович

Київ – 2026

Мета роботи:

Підготувати середовище збирання прошивки STM32, отримати повідомлення через UART у симуляторі Wokwi та перевірити роботу вбудованого світлодіода. Сформулювати призначення ExpoGuard, вимоги до майбутньої реалізації та план перевірки компонентів.

Постановка завдання:

Роботу виконано за формою B із використанням STM32 HAL та мови C. Згідно з методичкою ЛР1 підготовка охоплює систему збирання, запуск платформи, вибір компонентів і PRD [1].

1. Підготувати Docker і Makefile для збирання прошивки.
2. Перевірити конфігурацію USART2 та GPIO світлодіода.
3. Зібрати ELF, HEX і BIN та запустити прошивку у Wokwi.
4. Отримати стартове повідомлення й послідовні записи `tick`.
5. Описати тему ExpoGuard, датчики, функціональні вимоги та тести ЛР2.
6. Зберегти код і документацію в Git.

Середовище та платформа:

Середовище роботи: Ubuntu 24.04, Docker, STM32 HAL, Wokwi. Редактор проєкту: Visual Studio Code. Платформа: Nucleo-C031C6 з мікроконтролером STM32C031C6.

STM32C031C6 має ядро Cortex-M0+, 32 КБ Flash і 12 КБ RAM. Вбудований LED у моделі Wokwi під'єднаний до PA5 [2, 3]. Базову прошивку та конфігурацію перенесено зі SP-Boilerplate зі збереженням ліцензій. Його основа походить із прикладу Wokwi для STM32C031C6 [4]. У ЛР1 використано наявний UART-логер і цикл перемикання LED; драйвери датчиків належать до наступного етапу.

Хід роботи:

Підготовка проєкту:

Проєкт містить `docker/Dockerfile`, кореневий `Makefile`, каталог `firmware`, конфігурацію CubeMX `firmware.ioc`, схему `diagram.json` і файл `wokwi.toml`. Документацію зібрано в `docs`: PRD, опис компонентів, план лабораторних робіт і тестів.

Система збирання розділяє створення Docker-образа та компіляцію прошивки. Кореневий Makefile монтує каталог проєкту в `/workspace` і запускає `make -C firmware all` від імені користувача хоста. Це зберігає доступ користувача до створених файлів.

Збирання в Ubuntu:

У терміналі з каталогу `/home/barbars/ExpoGuard` виконано:
`make build`

Команда створила образ `stm32c0-wokwi-build:local` і запустила компіляцію. На рисунку 1 видно успішне створення та позначення образа, використання кешу й команду запуску контейнера.

```
barbars@Ubuntu24:~/ExpoGuard$ make build
docker build -t stm32c0-wokwi-build:local docker
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:22.04
 ---> b8b6ee6aa931
Step 2/4 : ARG DEBIAN_FRONTEND=noninteractive
 ---> Using cache
 ---> 2a3bade6f42e
Step 3/4 : RUN apt-get update && apt-get install -y --no-install-recommends     make     gcc-arm-none-eabi     bi
  && rm -rf /var/lib/apt/lists/*
 ---> Using cache
 ---> a03a80407cf5
Step 4/4 : WORKDIR /workspace
 ---> Using cache
 ---> 557e5bf48a35
Successfully built 557e5bf48a35
Successfully tagged stm32c0-wokwi-build:local
docker run --rm -v "/home/barbars/ExpoGuard:/workspace" -w /workspace \
        --user "1000:1000" stm32c0-wokwi-build:local \
        make -C firmware all
```

Рисунок 1 – Створення Docker-образа та запуск збирання прошивки

Результат компіляції:

Компілятор і компонувальник сформували `build/firmware.elf`; `arm-none-eabi-objcopy` створив `firmware.hex` і `firmware.bin`. Після завершення Make повернув керування командному рядку. Результат показано на рисунку 2.



Рисунок 2 – Завершення компіляції та створення ELF, HEX і BIN

Вивід `arm-none-eabi-size` містить:

Таблиця 1 – Розміри секцій прошивки

Секція або підсумок	Розмір, байт
`text`	13 464
`data`	120
`bss`	1 808
Разом `dec`	15 392

Сума `text + data` становить 13 584 байти; `data + bss` становить 1 928 байтів. Друга величина характеризує ці секції RAM, окремо від потреб стека та динамічної пам’яті.

Повторні чисті збірки:

Під час підготовки на macOS ARM64 через Docker Desktop виконано дві послідовності `make clean` та `make build`. Хеші SHA256 відповідних ELF, HEX і BIN збіглися. Логи та маніфести збережено у `docs/validation`. Цей результат стосується двох збірок тієї самої копії з тим самим Docker-образом.

Конфігурація UART та LED:

Таблиця 2 – Інтерфейси базової прошивки

Вузол	Налаштування
USART2	PA2 TX, PA3 RX, AF1; 115200 бод, 8 біт даних, 1 стоп-біт, без парності
LED	PA5, вихід GPIO; перемикання в основному циклі
I^2C1	PB8 SCL, PB9 SDA, AF6; ініціалізація наявна у базі

Відповідність PA2/PA3 та PB8/PB9 альтернативним функціям наведено в datasheet STM32C031, с. 35–36, таблиці 13 і 15 [2].

Виведення `printf` проходить через `_write` у `syscalls.c`, далі через `__io_putchar` до `HAL_UART_Transmit`. У базовій прошивці використано такий фрагмент:

```
int __io_putchar(int ch)
{
  uint8_t byte = (uint8_t)ch;
  if (HAL_UART_Transmit(&huart2, &byte, 1,
                        HAL_MAX_DELAY) != HAL_OK)
  {
    return EOF;
  }
  return ch;
}
```

Після ініціалізації виводиться стартове повідомлення. У циклі змінюється стан LED та друкується час роботи у секундах:

```
printf("STM32C031C6␣ready\r\n");

while (1)
{
  HAL_GPIO_TogglePin(Led_GPIO_Port, Led_Pin);
  printf("tick␣%lu\r\n",
         (unsigned long)(HAL_GetTick() / 1000U));
  HAL_Delay(1000);
}
```

Код задає затримку 1000 мс між перемиканнями. Повний цикл увімкнення й вимкнення LED складається з двох перемикань.

Запуск у Wokwi:

Файл `wokwi.toml` задає зібрану прошивку:

```
[wokwi]
version = 1
firmware = 'firmware/build/firmware.hex'
elf = 'firmware/build/firmware.elf'
```

USART2 з'єднано із Serial Monitor через PA2 та PA3. У Visual Studio Code запущено команду `Wokwi: Start Simulator`. На рисунку 3 видно плату Nucleo-C031C6, повідомлення `STM32C031C6 ready` та записи `tick 0`, `tick 1`, `tick 2`.

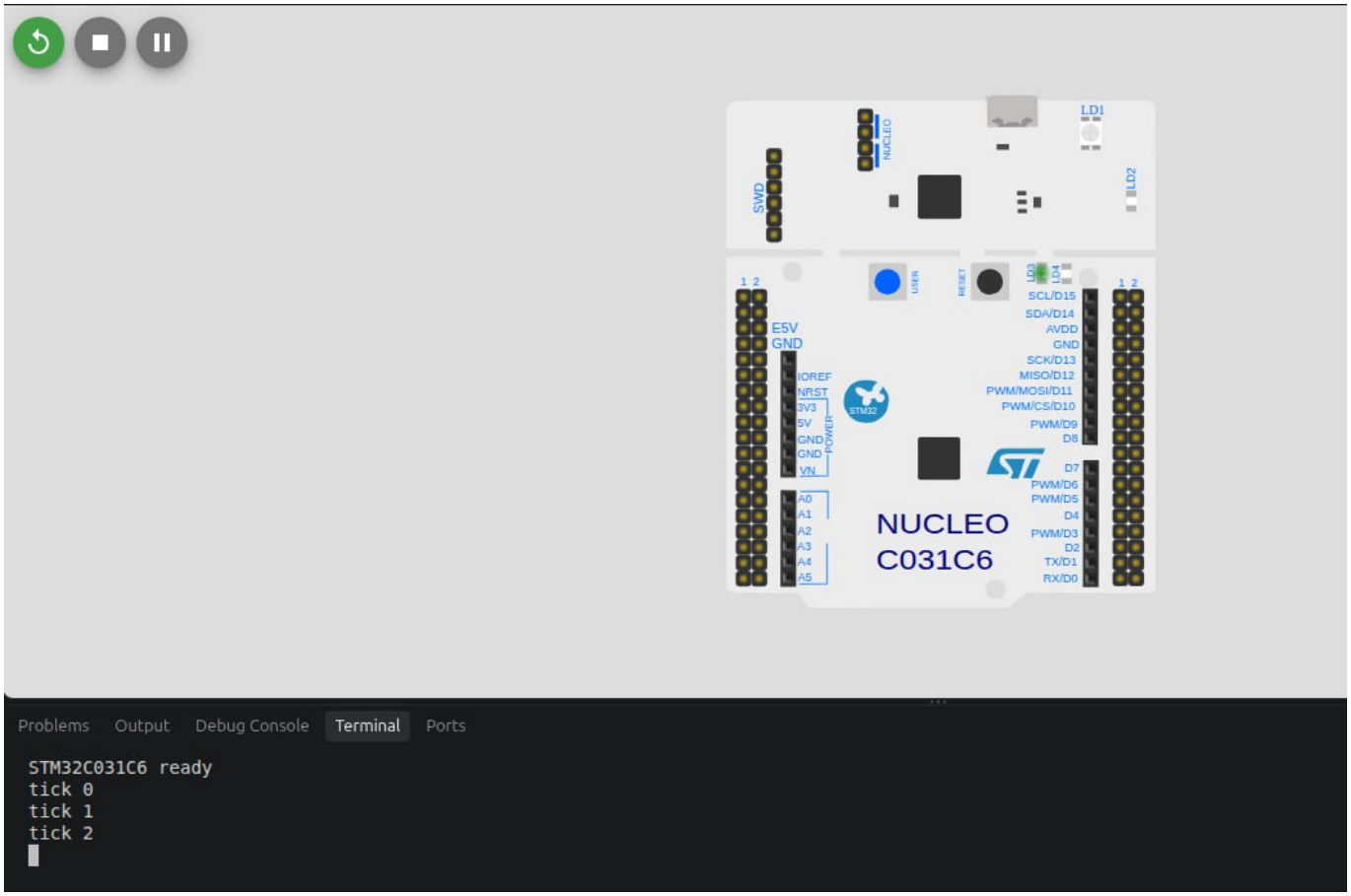


Рисунок 3 – Перший UART-вивід після запуску STM32C031C6 у Wokwi

Перевірка продовження виконання:

На наступному знімку журнал містить записи від `tick 0` до `tick 7`. Стан LED відрізняється від попереднього знімка. Разом із циклом `HAL_GPIO_TogglePin` це підтверджує періодичне виконання базової прошивки та зміну стану світлодіода.

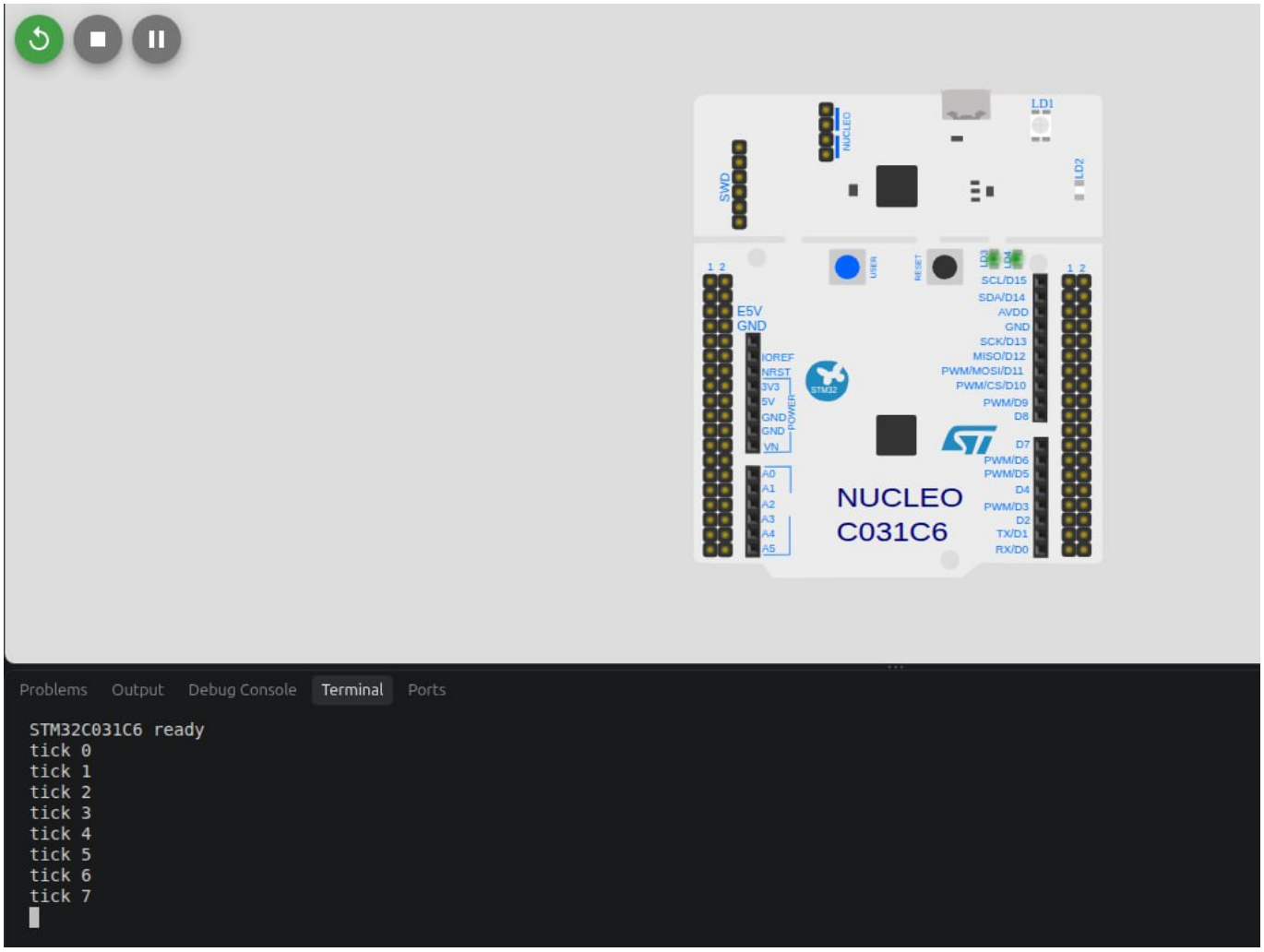


Рисунок 4 – Продовження UART-журналу до tick 7 та інший стан LED

Зауваження Docker:

На початку збирання Docker вивів повідомлення `DEPRECATED: The legacy builder is deprecated` і рекомендацію встановити компонент Buildx. Це повідомлення стосується механізму збирання образів. У показаному запуску legacy builder завершив збирання, після чого контейнер створив ELF, HEX і BIN. Перехід на BuildKit є окремим оновленням середовища [5].

Ідея проєкту та PRD:

ExpoGuard призначений для оператора виставки, який контролює стан макета. Проєкт розділяє рух відвідувачів поруч із макетом, механічний рух самого макета та зміну світла експозиції.

Оператор обиратиме один із трьох режимів:

- експозиція: звичайний показ макета з інформаційними подіями;
- охорона: механічний рух за межами встановленого порога створює тривогу;
- дозволене переміщення: погоджене перенесення макета без охоронної тривоги.

Подія PIR означає рух у зоні датчика. Зміна світла є окремим вимірюванням. Підрахунок людей, їхня ідентифікація та визначення причин втручання не входять до функцій ExpoGuard.

План перевірки вимог:

Для переходів режимів передбачено тест валідних і невідомих UART-команд. Для PIR перевірятимуть один перехід LOW→HIGH, повторний сигнал у межах HIGH та подію після повернення LOW. Для MPU6050 перевірятимуть спокій, значення нижче й вище порогів, калібрування та помилку обміну. Для LDR перевірятимуть крайні коди ADC, усереднення та гістерезис.

Спільний тест має підтвердити обробку трьох джерел і UART-команд без секундного блокування основного циклу. Перелік T-01–T-10 та відповідність вимогам збережено у `docs/test-plan.md`.

Функціональні вимоги:

PRD збережено у `docs/PRD.md`. Нижче наведено вимоги до реалізації ЛР2. Числові пороги й строки реакції є заданими критеріями майбутніх тестів.

FR-01: Після старту входити в експозицію; команди `mode expo`, `mode guard`, `mode move` змінюють режим із підтвердженням до 200 мс.

FR-02: Відхиляти невідому команду без зміни режиму.

FR-03: Реєструвати одну інформаційну подію PIR на перехід LOW→HIGH до 100 мс, без повторів протягом HIGH.

FR-04: Опитувати MPU6050 кожні 20 мс, перетворювати прискорення в g та кутову швидкість у градуси за секунду.

FR-05: В охороні активувати тривогу за відхилення прискорення понад 0,3 g або швидкості понад 30 градусів/с у двох послідовних вибірках.

FR-06: В експозиції та під час дозволеного переміщення реєструвати механічний рух без охоронної тривоги.

FR-07: Калібрувати базове положення за 100 стабільними вибірками; відхиляти калібрування під час руху.

FR-08: Опитувати LDR кожні 100 мс і виводити 12-бітний код ADC та відносний рівень світла.

FR-09: Усереднювати п’ять вибірок LDR; використовувати гістерезис 70/60% шкали ADC.

FR-10: Додавати до події час роботи в мс, режим, джерело та значення.

FR-11: Обробляти помилки I^2C з позначенням невалідних даних, зберігаючи обробку PIR, LDR і команд.

FR-12: Утримувати механічну тривогу до `ack`; приймати підтвердження після двох секунд спокою.

Документ PRD у проєкті:

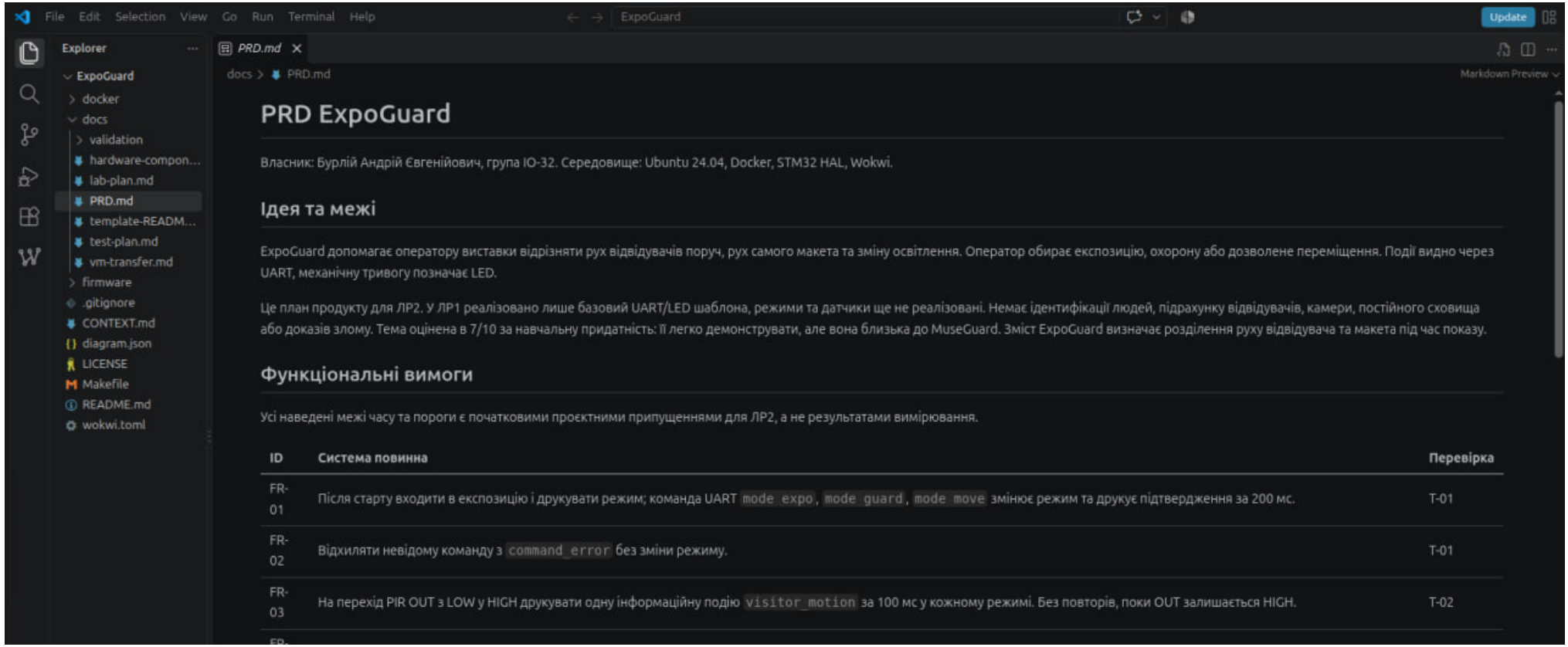


Рисунок 5 – PRD ExpoGuard у Visual Studio Code: призначення, власник і функціональні вимоги

Компоненти майбутньої реалізації:

Для ЛР2 обрано PIR через GPIO, MPU6050 через I²C і фоторезистор через ADC. У ЛР1 схема містить Nucleo-C031C6 та UART; датчики підключатимуться на наступному етапі.

PIR: рух відвідувача:

Модель `wokwi-pir-motion-sensor` має VCC, GND і цифровий OUT. Типово вихід HIGH триває п’ять секунд, інтервал блокування після LOW — 1,2 с. Повторний рух може подовжувати HIGH [6]. PIR реєструватиме рух поруч із макетом як інформаційну подію. Кнопка давала б ручну команду, ультразвуковий датчик — відстань до об’єкта.

MPU6050: рух макета:

Модель `wokwi-mpu6050` містить тривісний акселерометр і гіроскоп. Інтерфейс: SDA та SCL; адреса `0x68` за AD0 у LOW або `0x69` за AD0 у HIGH. Прискорення задається в g, кутова швидкість — у градусах за секунду [7]. MPU6050 дає обидва виміри механічного руху. Контакт нахилу дає дискретний стан; акселерометр без гіроскопа не вимірює кутову швидкість окремо.

Фоторезистор: світло експозиції:

Модель `wokwi-photoresistor-sensor` містить LDR і резистор 10 кОм. AO дає аналоговий сигнал світла; DO — цифровий вихід [8]. Для ExpoGuard обрано AO через ADC. Потенціометр давав би ручний рівень; цифровий I²C-датчик світла не забезпечував би окремого ADC-інтерфейсу.

План інтерфейсів:

База вже налаштовує I²C1 на PB8 SCL та PB9 SDA. MPU6050 планується підключити до цих контактів, AD0 — до GND. Для PIR розглянуто PA4, для LDR — PA0; GPIO й ADC налаштовуватимуться в ЛР2.

У Wokwi планується живлення від 3V3 зі спільною землею. Характеристики фізичних модулів перевіряються за документацією їхнього виробника. Код ADC використовуватиметься як відносний вимір світла.

Git та документація:

Робоча гілка проєкту: `feature/lab-1`. Гілка `main` містить початковий коміт із порожнім деревом; поверх нього сформовано чотири змістовні коміти підготовки.

Таблиця 3 – Локальні коміти проєкту

Коміт	Вміст
`fca78ec`	Початкова порожня історія
`48ff001`	База STM32 HAL, Docker, UART і LED
`b95ce2e`	Словник ExpoGuard і функціональні вимоги
`e7af610`	План лабораторних робіт, компонентів і перевірок
`90102c2`	Логи чистих збірок та передача копії в VM

README описує збирання та запуск, `CONTEXT.md` — терміни. У `docs` збережено PRD, опис компонентів, план лабораторних робіт і тести. `firmware/build` виключено з Git.

Репозиторій: https://github.com/AndriiBurlii/ExpoGuard. Ідентифікатори в таблиці відповідають перевіреній локальній історії.

Технічні рішення під час підготовки:

1. **Відповідність цільовому MCU.** Збережено STM32C031C6, його HAL, startup і linker script; збирання використовує `-mcpu=cortex-m0plus`.
2. **Маршрут UART.** Узгоджено PA2 TX і PA3 RX із Serial Monitor; `printf` працює через newlib-hook та HAL.
3. **Доступ до build-файлів.** Docker запускає збирання з UID/GID користувача; каталог проєкту змонтовано в контейнер.
4. **Повторюваність збирання.** Перед кожною з двох перевірок очищено build; отримані SHA256 порівняно для трьох артефактів.
5. **Попередження Docker.** Попередження legacy builder не перервало збирання: створено ELF, HEX і BIN.

Висновки:

У роботі підготовлено ExpoGuard на базі STM32C031C6 зі STM32 HAL. Docker-збирання створило ELF, HEX та BIN; для прошивки отримано значення `text = 13464`, `data = 120`, `bss = 1808` байтів.

У Wokwi отримано стартове повідомлення `STM32C031C6 ready` та послідовність `tick 0`–`tick 7`. Знімки симуляції фіксують різні стани LED; код основного циклу задає перемикання після затримки 1000 мс.

Під час аналізу базової прошивки розглянуто ініціалізацію HAL, GPIO та USART2, передавання `printf` через UART і зв’язок зібраних файлів із конфігурацією Wokwi. Для ExpoGuard сформульовано дванадцять функціональних вимог і тести їхнього приймання.

Наступний етап: підключення PIR, MPU6050 та LDR, реалізація трьох режимів, калібрування й фільтрація вимірювань. Для спільного опитування компонентів секундну затримку базового циклу буде замінено неблокуючим розкладом за часом роботи.

Список використаних джерел:

[1] *Лабораторна робота №1. Налаштування середовища розробки та bring-up STM32-платформи*. Методичка LR1.pdf, розділи 2.2, 3 і 4. Локальну копію збережено в матеріалах документа.

[2] STMicroelectronics. *STM32C031x4/x6. Datasheet DS13867 Rev 4*. С. 1, 35 і 36. URL: https://www.st.com/resource/en/datasheet/stm32c031c6.pdf (дата зверн. 04.10.2026).

[3] Wokwi. *STM32 Nucleo64 C031C6. Onboard LED*. URL: https://docs.wokwi.com/parts/board-st-nucleo-c031c6 (дата зверн. 04.10.2026).

[4] Wokwi. *STM32 Hello Wokwi*. URL: https://github.com/wokwi/stm32-hello-wokwi (дата зверн. 04.10.2026).

[5] Docker. *Docker Build Overview*. URL: https://docs.docker.com/build/concepts/overview/ (дата зверн. 04.10.2026).

[6] Wokwi. *PIR motion sensor. Pin names, Attributes, Using the sensor*. URL: https://docs.wokwi.com/parts/wokwi-pir-motion-sensor (дата зверн. 04.10.2026).

[7] Wokwi. *MPU6050 6-Axis Accel and Gyro Sensor. Pin names, Units*. URL: https://docs.wokwi.com/parts/wokwi-mpu6050 (дата зверн. 04.10.2026).

[8] Wokwi. *Photoresistor sensor. Pin names, Operation*. URL: https://docs.wokwi.com/parts/wokwi-photoresistor-sensor (дата зверн. 04.10.2026).